Hands-on 2

Hibernate XML Config Implementation — Walkthrough

SME explanation of the sample Hibernate implementation referenced from TutorialsPoint (Hibernate Examples). Covers Object/Relational mapping via XML configuration, and the end-to-end session/transaction lifecycle used to perform CRUD operations.

Topics Covered

- How Object-to-Relational mapping is done in the Hibernate XML configuration file
- SessionFactory, Session, and Transaction — roles and lifecycle
- beginTransaction(), commit(), and rollback()
- session.save(), session.createQuery().list(), session.get(), session.delete()

1. Object-to-Relational Mapping in the Hibernate XML Configuration

Hibernate uses **two separate XML files** to achieve Object/Relational Mapping (ORM), and it's important to understand what each one is responsible for.

a) The Mapping File — Employee.hbm.xml

This file tells Hibernate how a Java class corresponds to a database table, field by field.

```
<hibernate-mapping>
   <class name="com.example.Employee" table="EMPLOYEE">
      <id name="id" type="int" column="id">
         <generator class="native"/>
      </id>
      <property name="firstName" column="first_name" type="string"/>
      <property name="lastName" column="last_name" type="string"/>
      <property name="salary" column="salary" type="int"/>
   </class>
</hibernate-mapping>
```

- **<class name=... table=...>** — maps the Java class to a specific database table.
- **<id>** — maps the primary key field. The nested **<generator>** tag tells Hibernate *how* to generate that key (e.g. **native** lets the database decide; other options include **increment**, **identity**, **sequence**, **uuid**).
- **<property>** — maps each Java field to a table column. **name** is the Java field, **column** is the database column (if omitted, Hibernate assumes they match), and **type** is the Hibernate type, which itself maps to the correct JDBC/SQL type.

In short: **class** → **table**, **fields** → **columns**. Hibernate reads this file at runtime and uses it to translate object operations (save/get/update/delete) into the correct SQL.

b) The Configuration File — hibernate.cfg.xml

This is a separate concern from mapping — it defines **how to connect** to the database and **global settings** for the SessionFactory.

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.dialect">
        org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.connection.driver_class">
        com.mysql.cj.jdbc.Driver</property>
    <property name="hibernate.connection.url">
        jdbc:mysql://localhost/test</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">pass123</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

    <mapping resource="Employee.hbm.xml"/>
  </session-factory>
</hibernate-configuration>
```

- **Connection properties** — driver class, JDBC URL, username, password.
- **dialect** — tells Hibernate which SQL flavor to generate (MySQL, Oracle, PostgreSQL, etc.), since SQL syntax varies slightly across databases.
- **hbm2ddl.auto** — controls whether Hibernate auto-creates/updates tables from the mapping file (**update**, **create**, **validate**, etc.) — convenient in development, riskier in production.
- **<mapping resource=...>** — the link that pulls each **.hbm.xml** file into this SessionFactory so Hibernate knows which class-to-table mappings belong to it.

Summary: the **.hbm.xml** file defines *what* to map, and **hibernate.cfg.xml** defines *where* to connect and *which* mappings to load.

2. End-to-End Operation Flow

The table below shows the order these components/methods are typically invoked in a driver class such as **ManageEmployee.java**, followed by a detailed explanation of each.

SessionFactory

```
SessionFactory factory = new Configuration()
        .configure()
        .buildSessionFactory();
```

- Built **once per application** — it is heavyweight: it reads **hibernate.cfg.xml**, parses all mapping files, and sets up connection pooling.
- **Thread-safe** and immutable once created. Think of it as the factory that manufactures **Session** objects.
- Typically created as a singleton at application startup.

Session

```
Session session = factory.openSession();
```

- A lightweight, **non-thread-safe** object representing a single unit of work / conversation with the database.
- Wraps a JDBC connection and is the main API for CRUD operations (save, get, update, delete, query).
- Open one per logical task (e.g. per request or per method) and close it when done — unlike the SessionFactory, it is not kept around.

Transaction

```
Transaction tx = null;
```

- Represents a unit of work that should be **atomic** — either all the database changes happen, or none do.
- Hibernate's Transaction wraps the underlying JDBC/JTA transaction, so application code doesn't need to know which one is in use underneath.

beginTransaction()

```
tx = session.beginTransaction();
```

- Starts a new transaction on the current session.
- Nothing done afterward (save/update/delete) is actually persisted to the database until **commit()** is called — it is held in the session cache until then.

commit()

```
tx.commit();
```

- Flushes all pending changes to the database and finalizes the transaction.
- Only after commit() do save/update/delete operations become permanent.

rollback()

```
try {
   tx = session.beginTransaction();
   // ... do work ...
   tx.commit();
} catch (Exception e) {
   if (tx != null) tx.rollback();
   e.printStackTrace();
} finally {
   session.close();
}
```

- Used inside a **catch** block — if anything goes wrong during the transaction (an exception is thrown), the transaction is rolled back so the database is not left in a half-changed, inconsistent state.

session.save()

```
Employee emp = new Employee("Zara", "Ali", 5000);
Integer empId = (Integer) session.save(emp);
```

- Inserts a new record into the database corresponding to the given object.
- Returns the generated identifier (useful with auto-generated keys like **native** or **increment**).

- The insert is only *staged* — it isn't guaranteed to hit the database until the transaction commits (depending on flush mode).

session.createQuery().list()

```
Query query = session.createQuery("FROM Employee");
List<Employee> employees = query.list();
```

- This uses **HQL (Hibernate Query Language)** — note the query is against the **class name** (Employee), not the table name (EMPLOYEE). Hibernate translates it into actual SQL using the mapping file.
- **.list()** executes the query and returns the full result set as a **List** of entity objects — Hibernate automatically converts each row back into an Employee object.
- This is the object-oriented way to do bulk reads / "SELECT *"-style operations.

session.get()

```
Employee emp = (Employee) session.get(Employee.class, empId);
```

- Retrieves a **single record by its primary key**.
- Requires the class and the ID — Hibernate generates the equivalent of **SELECT ... WHERE id = ?**.
- If no row matches, **get()** returns **null** (unlike **load()**, which throws an exception if the row isn't found — a common distinction to remember).

session.delete()

```
session.delete(emp);
```

- Deletes the row corresponding to the given persisted object.
- Typically the object is fetched with **get()** first, then passed to **delete()** — Hibernate uses the object's identifier to generate the **DELETE ... WHERE id = ?** statement.
- Like save(), this change is staged until the transaction commits.

3. Putting It All Together — Full Round Trip

A complete end-to-end flow using all the pieces above, in the order they are normally invoked:

```
SessionFactory factory = new Configuration()
        .configure().buildSessionFactory();
Session session = factory.openSession();
Transaction tx = null;

try {
   tx = session.beginTransaction();

   // Create
   Employee emp = new Employee("Zara", "Ali", 5000);
   Integer empId = (Integer) session.save(emp);

   // Read all
   List<Employee> employees =
        session.createQuery("FROM Employee").list();

   // Read one
   Employee e = (Employee) session.get(Employee.class, empId);
```

```
   // Delete
   session.delete(e);

   tx.commit();
} catch (Exception ex) {
   if (tx != null) tx.rollback();
   ex.printStackTrace();
} finally {
   session.close();
}
```

Lifecycle Summary

Component	Scope	Purpose
SessionFactory	Once per application	Reads config + mappings; produces Sessions
Session	Per unit of work	Main API for CRUD + query operations
Transaction	Per session (per task)	Groups operations into an atomic unit
commit() / rollback()	End of transaction	Finalizes or reverts pending changes

---

Reference: TutorialsPoint — Hibernate Examples (tutorialspoint.com/hibernate/hibernate_examples.htm)